

Review II

E-Commerce Order Management System

Presented by

AVULA VENKATA NAVEEN(25B11AI075)

NAKKA KRANTHI(25B11AI606)

NEETIPALLI SATYA SAI HANUMA PHANINDRA(25B11AI839)

SATYA NANDA KISHORE GOPI(25B11AIA79)

Presentation Outline

1 Project Overview



- Domain & Problem Statement
- Objectives
- System Overview
- Entities & Attributes
- ER Diagram & Relationships
- Relational Database Schema
- Primary & Foreign Keys
- GitHub & Team

2 PostgreSQL Implementation



- Database & Table Creation (DDL)
- Sample Data Insertion (DML)
- DQL Queries

3 SQL Operations



- CRUD Operations
- Basic Retrieval Queries (SELECT)
- Filtering, Sorting & Grouping
- Join Queries
- Aggregate Functions

4 Advanced SQL Queries



- Nested / Correlated Subqueries
- Views and Set Operations (UNION, INTERSECT, EXCEPT)
- Stored Procedures / Functions
- Triggers

5 Data Integrity & Security



- Primary & Foreign Key Constraints
- NOT NULL, UNIQUE & CHECK Constraints
- Referential Integrity (with example)
- DCL – GRANT & REVOKE
- TCL – COMMIT, ROLLBACK & SAVEPOINT

6 Normalization



- UNF → 1NF → 2NF → 3NF
- Redundancy Reduction
- Update / Insert / Delete Anomalies

7 Reports & Sample Outputs



- Order Details Report
- Customer Spending Report
- Product Sales Report
- Product Stock Report
- Screenshots of Query Outputs

8 Documentation & Live Demonstration



- Project Documentation (PPT + Report)
- PostgreSQL Live Demonstration
- Key Features and Results

9 Conclusion



- Results Achieved
- Project Benefits
- Future Enhancements
- Conclusion





E-COMMERCE ORDER MANAGEMENT SYSTEM

A PostgreSQL-based database for managing customers, addresses, products, categories, orders, order items and payments.



PROBLEM STATEMENT

- Redundant customer and product information
- Difficult order and payment tracking
- Manual sales calculations
- Inconsistent data without proper integrity rules



OBJECTIVES

- Build a normalized database up to 3NF
- Maintain reliable relationships using primary and foreign key constraints
- Support CRUD operations
- Enable reporting and analytical SQL queries

ENTITIES & ATTRIBUTES

1

Customer



- **customer_id (PK)**
- name
- email
- registration_date

2

Customer_Phone



- **phone_id (PK)**
- customer_id (FK)
- phone

3

Address



- **address_id (PK)**
- customer_id (FK)
- street
- city
- state
- pincode

4

Category



- **category_id (PK)**
- category_name
- parent_category_id (FK)

5

Product



- **product_id (PK)**
- product_name
- price
- stock_quantity
- category_id (FK)

6

Product_Tags



- product_id (FK)
- **tag (PK)**

7

Orders



- **order_id (PK)**
- customer_id (FK)
- address_id (FK)
- order_date
- order_status

8

Order_Item



- **item_id (PK)**
- order_id (FK)
- product_id (FK)
- quantity
- unit_price

9

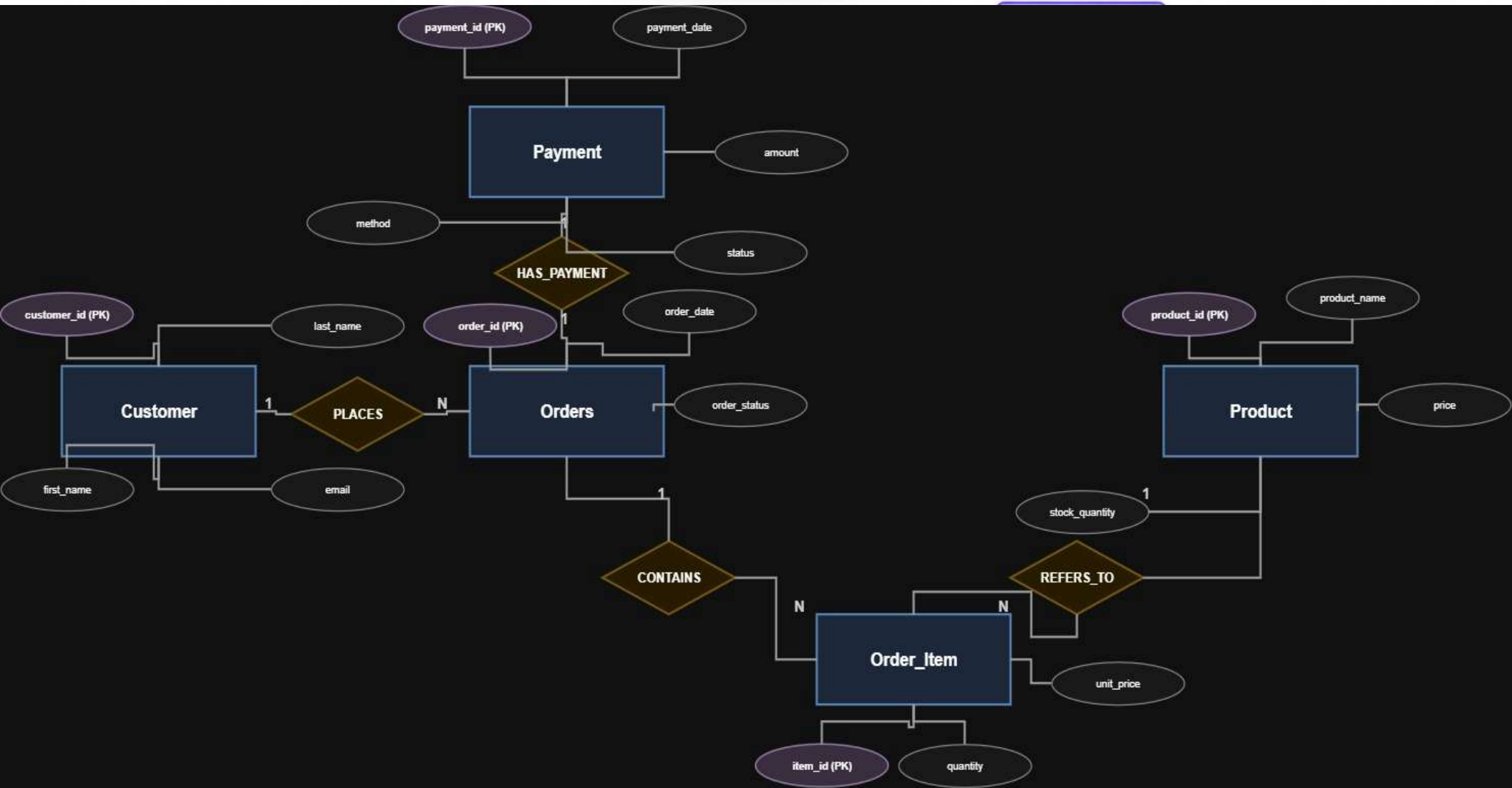
Payment



- **payment_id (PK)**
- order_id (FK)
- amount
- method
- status
- transaction_id

ER DIAGRAM

E-COMMERCE ORDER MANAGEMENT SYSTEM



E-Commerce Order Management System — Core Conceptual ER Diagram

Five core entities shown for presentation clarity. Detailed entities such as Address, Category and Product_Tags remain in the complete database design.

Rectangle = Entity • Diamond = Relationship • Oval = Attribute • Purple Oval = Primary Key



CUSTOMER & CATALOG

1 CUSTOMER

CUSTOMER(customer_id PK, first_name, last_name, email, registration_date)

2 CUSTOMER_PHONE

CUSTOMER_PHONE(phone_id PK, customer_id FK, phone)

3 ADDRESS

ADDRESS(address_id PK, customer_id FK, street, city, state, pincode, address_type)

4 CATEGORY

CATEGORY(category_id PK, category_name, parent_category_id FK)

5 PRODUCT

PRODUCT(product_id PK, product_name, description, price, stock_quantity, category_id FK)

6 PRODUCT_TAGS

PRODUCT_TAGS(product_id PK/FK, tag PK)



ORDER & PAYMENT

1 ORDERS

ORDERS(order_id PK, customer_id FK, address_id FK, order_date, order_status)

2 ORDER_ITEM

ORDER_ITEM(item_id PK, order_id FK, product_id FK, quantity, unit_price)

3 PAYMENT

PAYMENT(payment_id PK, order_id FK/UNIQUE, payment_date, amount, method, status, transaction_id)



The relational schema is the table-level implementation derived from the conceptual ER model.

PRIMARY & FOREIGN KEYS



PRIMARY KEYS

Each table has a unique primary key to uniquely identify records.

S.No.	Table	Primary Key
1	Customer	customer_id
2	Customer_Phone	phone_id
3	Address	address_id
4	Category	category_id
5	Product	product_id
6	Orders	order_id
7	Order_Item	item_id
8	Payment	payment_id
9	Product_Tags	(product_id, tag)



FOREIGN KEYS

Foreign keys establish relationships between tables and enforce referential integrity.

S.No.	Foreign Key	References	Description
1	Customer_Phone.customer_id	Customer(customer_id)	Links phone to customer
2	Address.customer_id	Customer(customer_id)	Links address to customer
3	Orders.customer_id	Customer(customer_id)	Links order to customer
4	Orders.address_id	Address(address_id)	Links order to delivery address
5	Product.category_id	Category(category_id)	Links product to category
6	Order_Item.order_id	Orders(order_id)	Links order item to order
7	Order_Item.product_id	Product(product_id)	Links order item to product
8	Payment.order_id	Orders(order_id)	Links payment to order
9	Product_Tags.product_id	Product(product_id)	Links tag to product



Primary keys ensure uniqueness, and foreign keys enforce referential integrity between related tables.

DATABASE & CONSTRAINTS



POSTGRESQL IMPLEMENTATION



9 project tables
created in PostgreSQL.



GENERATED ALWAYS AS IDENTITY
for automatic IDs.



PRIMARY KEY
for unique entity identification.



FOREIGN KEY
for referential integrity.



DATA INTEGRITY CONSTRAINTS



NOT NULL
for mandatory attributes.



UNIQUE
for email, category names and transaction IDs.



CHECK
constraints for price, stock, quantity and payment amount.



DEFAULT
values for dates and initial status.



Execution Environment: DB Fiddle — PostgreSQL

DDL, DML & DQL OPERATIONS



DDL – DATA DEFINITION LANGUAGE

Used to define and manage the database structure.
In this project, the following constraints are used:



CREATE TABLE

to create tables (9 tables created)



PRIMARY KEY

for unique entity identification



FOREIGN KEY

for referential integrity



UNIQUE

for email, category names and transaction IDs



CHECK

for price, stock, quantity and payment amount



DEFAULT

for dates and initial status values



GENERATED ALWAYS AS IDENTITY

for automatic ID generation



DML – DATA MANIPULATION LANGUAGE

Used to insert, update and delete data in the tables.

Examples:

```
-- Insert
INSERT INTO Product (product_name, price, stock_quantity, category_id)
VALUES ('Laptop', 75000, 10, 1);

-- Update
UPDATE Orders SET order_status = 'Shipped' WHERE order_id = 1;

-- Delete
DELETE FROM Customer_Phone WHERE phone_id = 5;
```



DQL – DATA QUERY LANGUAGE

Used to retrieve and analyze data from the database.

Examples:

```
-- Select all products
SELECT * FROM Product;

-- Select order details
SELECT * FROM Order_Details;

-- Select products by category (Join)
SELECT p.product_id, p.product_name, c.category_name
FROM Product p
JOIN Category c ON p.category_id = c.category_id
ORDER BY p.product_name;
```



INTEGRITY CONTROLS

Constraints and triggers ensure that the data is accurate, consistent and relationship-safe.



PRIMARY KEY

Prevents duplicate identifiers for each table.



FOREIGN KEY

Maintains referential integrity and prevents orphan records.



UNIQUE

Prevents duplicate emails, category names and transactions.



NOT NULL

Ensures mandatory attributes are not empty.



CHECK

Rejects invalid price, stock, quantity and payment amounts.



DEFAULT

Provides safe initial dates and status values.



TRIGGER (Order_Item)

Checks available stock before insertion into Order_Item.



EXAMPLE CONSTRAINTS & TRIGGER

PRIMARY KEY (Example)

```
CREATE TABLE Product (  
  product_id INT GENERATED ALWAYS AS ID-  
ENTITY PRIMARY KEY,  
  product_name VARCHAR(100) NOT NULL  
);
```

FOREIGN KEY (Example)

```
CREATE TABLE Order_Item (  
  order_id INT NOT NULL,  
  product_id INT NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES Product(product_id)  
);
```

UNIQUE (Example)

```
CREATE TABLE Customer (  
  email VARCHAR(100) UNIQUE,  
  phone_number VARCHAR(15) UNIQUE  
);
```

CHECK (Example)

```
CREATE TABLE Product (  
  price NUMERIC(10,2) CHECK (price >= 0),  
  stock_quantity INT CHECK (stock_quantity >= 0)  
);
```

DEFAULT (Example)

```
CREATE TABLE Orders (  
  order_date TIMESTAMP DEFAULT CURRENT_I  
TIMESTAMP,  
  order_status VARCHAR(20) DEFAULT  
  'Pending'  
);
```

TRIGGER (Order_Item — Stock Validation)

```
CREATE OR REPLACE FUNCTION validate_stock()  
RETURNS TRIGGER AS $$  
BEGIN  
  -- check available stock before insert  
  -- (implementation in project code)  
END;  
$$ LANGUAGE plpgsql;
```



Result: Consistent, valid and relationship-safe data.



REVIEW II DOCUMENTATION STATUS

The complete project documentation covers the following:



ABSTRACT, INTRODUCTION & PROBLEM STATEMENT

Overview of the project and motivation.



OBJECTIVES & TOOLS/TECHNOLOGIES

Project goals and tools used (PostgreSQL, DB Fiddle).



ENTITIES, ATTRIBUTES & RELATIONSHIPS

Description of main entities and their attributes.



ER DIAGRAM WITH CARDINALITIES

Complete ER diagram showing relationships.



RELATIONAL DATABASE SCHEMA

Table structures with constraints.



NORMALIZATION UP TO 3NF

Stepwise normalization with justification.



POSTGRESQL DDL, DML AND DQL

CREATE, INSERT, UPDATE, DELETE, SELECT queries.



CRUD, JOINS, AGGREGATES, SUBQUERIES AND VIEWS

Implementation with sample queries and outputs.



STORED PROCEDURE & STOCK-VALIDATION TRIGGER

Update order status and check available stock.



DATA INTEGRITY AND VALIDATION

Primary key, foreign key, unique, not null, check, default constraints and trigger validation.



SCREENSHOTS / SAMPLE OUTPUTS

Execution screenshots from DB Fiddle.



Final member details, GitHub URL and actual execution screenshots are to be inserted.

CONCLUSION



CONCLUSION

The E-Commerce Order Management System provides a normalized PostgreSQL solution for managing customers, products, orders and payments.



KEY LEARNINGS



ER modeling and relational schema design



Normalization up to 3NF



PostgreSQL DDL, DML, DQL and CRUD



Joins, aggregates, subqueries and views



Constraints, triggers and stored procedures



FUTURE ENHANCEMENTS

Customer/admin UI, authentication and roles, shipment tracking, inventory reservation, invoices and dashboard analytics.



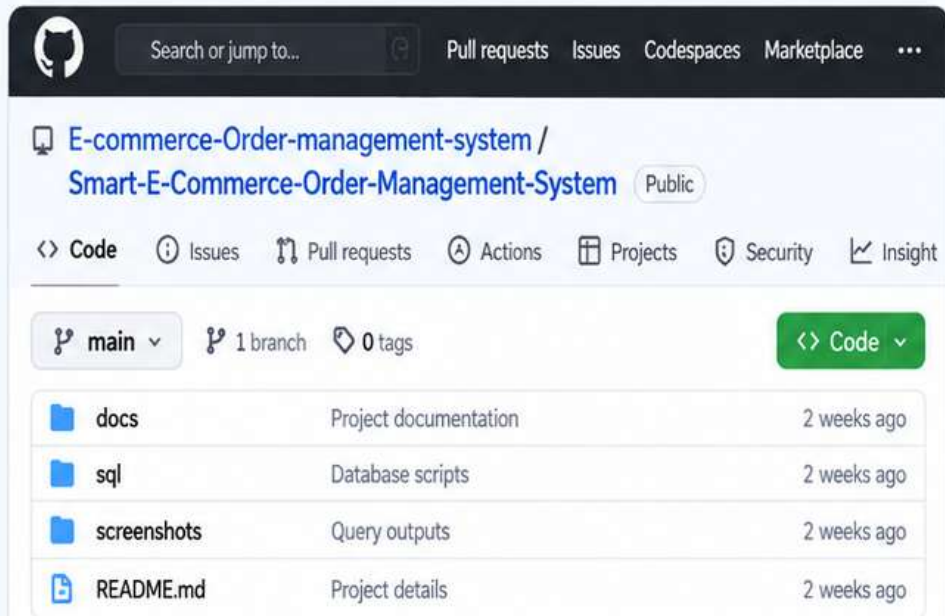
GITHUB REPOSITORY

The source code, database scripts, documentation and project files are maintained in the following GitHub repository.



Repository URL:

<https://github.com/E-commerce-Order-management-system/Smart-E-Commerce-Order-Management-System>



TEAM RESPONSIBILITIES

Each team member contributed to different modules of the project as listed below.

Member	Name	Roll Number	Responsibilities
1	Phanindra	25B11AI839	ER design, normalization and relational schema
2	Kranthi	25B11AI606	PostgreSQL DDL, constraints and sample data
3	Kishore	25B11AIA79	SQL queries, views, procedure and trigger
4	Naveen	25B11AI075	Testing, screenshots, documentation and presentation



THANK YOU

**E-Commerce Order
Management System**

Questions & Discussion